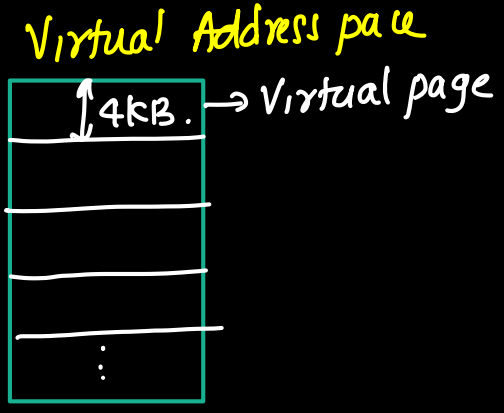
Virtual Memory (Note)

- Consider a program of size 32 GB stored on the hard disk. But the available RAM is 16 GB.
- Since the program size is larger than RAM, the entire program cannot be loaded into main memory at once.
→ without special support, such program cannot be executed. this problem can be solved using the concept of virtual memory
- Virtual Memory is a memory management technique that uses secondary storage as an extension of main memory. It provides the illusion of a large memory space to programs, even though available main memory is limited.
- When the operation started, the operating system created virtual address space for the program.
→ Virtual address space consists of virtual address that can be used by the program.
→ Virtual address space is divided into fixed sized blocks are called virtual pages. (typically size 4KB)



→ Operating s/m knows the mapping b/w virtual pages and disc location of executable file.

→ During execution, CPU generates Virtual Addresses. but CPU need to know the main memory address (physical address) to access the main memory. So Virtual address are translated into physical Address . this is called address translation.

→ CPU access the main memory using the physical address obtained. If required page (information) is present in the MM, then CPU access the information and execute it.

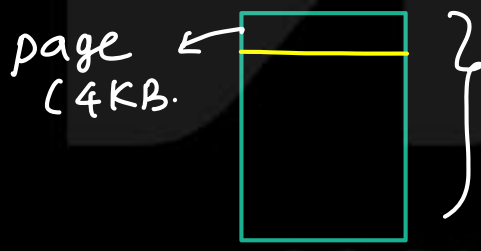
→ If required page is not present in the main memory then page fault occurs, then operating sys loads the required page from hard disk to into RAM and resume execution.

→ Thus Virtual memory allows the programs larger than physical memory to execute efficiently loading only required portions of the program into main memory.

Virtual Address.

- programs access virtual memory using virtual address
- → Virtual memory is divided into fixed sized blocks (typically 4KB). called virtual pages.
- → Virtual Address have two fields
  1. page number field
  2. page offset
- → Virtual address is divided into two fields
  1. Virtual page Number (VPN)
  2. page offset.

Eg: Virtual memory size = 2GB.



1Byte = 1word

No: of pages in the Virtual Memory (VM) = $\frac{\text{VM size}}{\text{page size}}$

$$n = \frac{2GB}{4KB} = \frac{2 \times 2^{30} \text{ word}}{4 \times 2^{10} \text{ word}} = \frac{2^{31}}{2^{12}} = 2^{19}$$

No: of bits to represent a page = $\log_2$ no: of pages

page no bits = $\log_2 2^{19} = 19$ bit

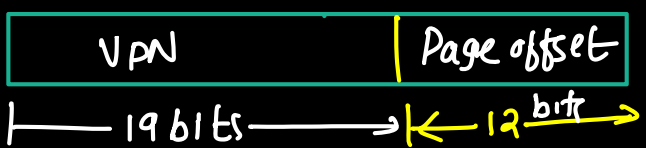
Virtual page no bits = 19 bits.

page size 4KB = $2^2 \times 2^{10} = 2^{12}$

No: of bits to represent words inside the page = $\log_2 2^{12}$

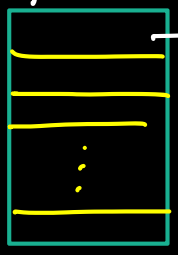
page offset = 12 bits

Virtual Address.



Physical Address

- these are used to represent main memory locations
- → Main memory is divided in fixed sized pages called physical page
- →



physical page size = Virtual page size (eg: 4KB)

- physical Address have two field
  1. physical page number
  2. page offset

physical page no:	page offset

eg: Main Mem size = 128MB = $2^7 \times 2^{20} = 2^{27}$

page size = 4KB = $2^2 \times 2^{10} = 2^{12}$

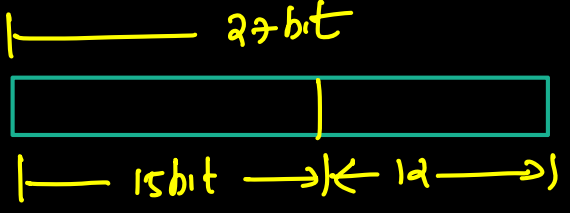
no: of page = $\frac{2^{27}}{2^{12}} = 2^{15}$

no: of bits to represent page = $\log_2 2^{15}$ = 15bit

no: of bits to represent words inside the page = page offset

$= \log_2(\text{page size})$

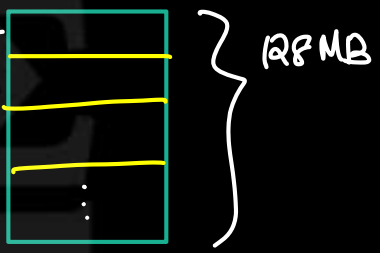
$= \log_2 2^{12} = 12$ bits



Physical Memory.

- main memory referred to as physical memory.
- physical memory is divided into pages called physical pages
- physical memory page size = Virtual memory page size.
- → physical memory accessed using physical Address
- physical Address have two field
  1. physical page no (PPN)
  2. page offset

eg: Physical Memory size = 128 MB



$$\text{no: of pages} = \frac{128\ MB}{4KB} = \frac{2^7 \times 2^{20}}{2^2 \times 2^{10}} = \frac{2^{27}}{2^{12}} = 2^{15}$$

no: of bits to represent page = physical page no

page offset of physical memory = page offset of Virtual Memory.

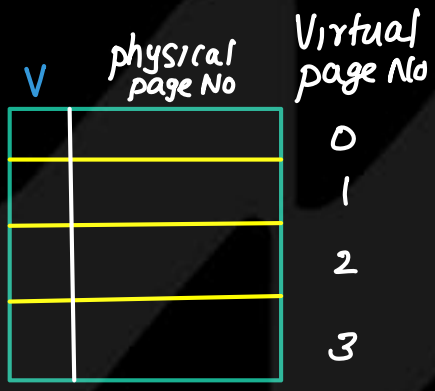
page table (Note)

— processor uses a page table to convert virtual address to physical address.

→ Page table contains an entry for each virtual page.

eg: If Virtual Address space have 4 virtual page with number VP_0, VP_1, VP_2, VP_3, then page table have 4 entries.

→ Each entry in the page table contains a physical page number and valid bit.

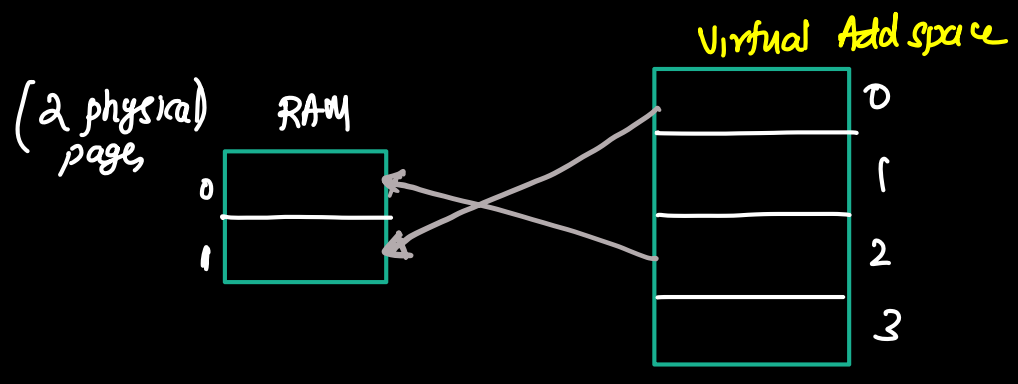


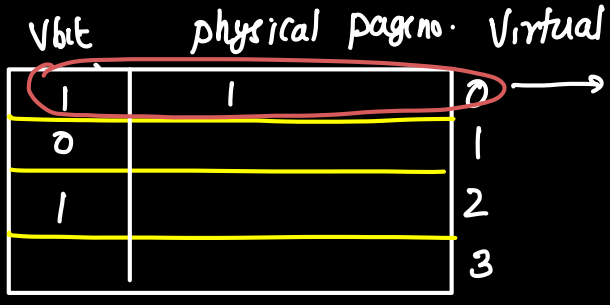
— If the valid bit is 1, then virtual page maps to the physical page in the entry (require information is available in the physical memory

→ If the valid bit is 0, then virtual page is not found in the main memory.

eg: Main memory have two pages (2 physical pages.

Virtual address space have 4 virtual pages. (0, 1, 2, 3)



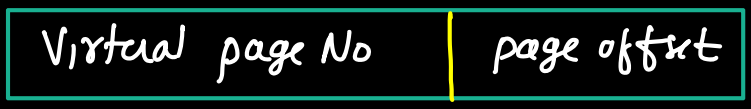


Valid bit = 1, Virtual page no is mapped to physical page no 1. ie required information is present in the physical memory

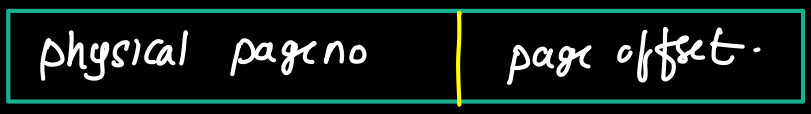
→ page table stored in the main Memory.

Address translation using page table. (Note)

— Virtual Address format



— physical Address format



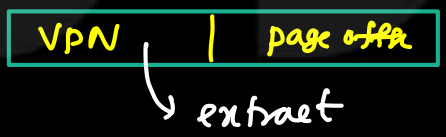
→ page offset for the both the address is same no need to translate

→ Virtual page number needs to be translated.

Steps

① CPU generate virtual Address.

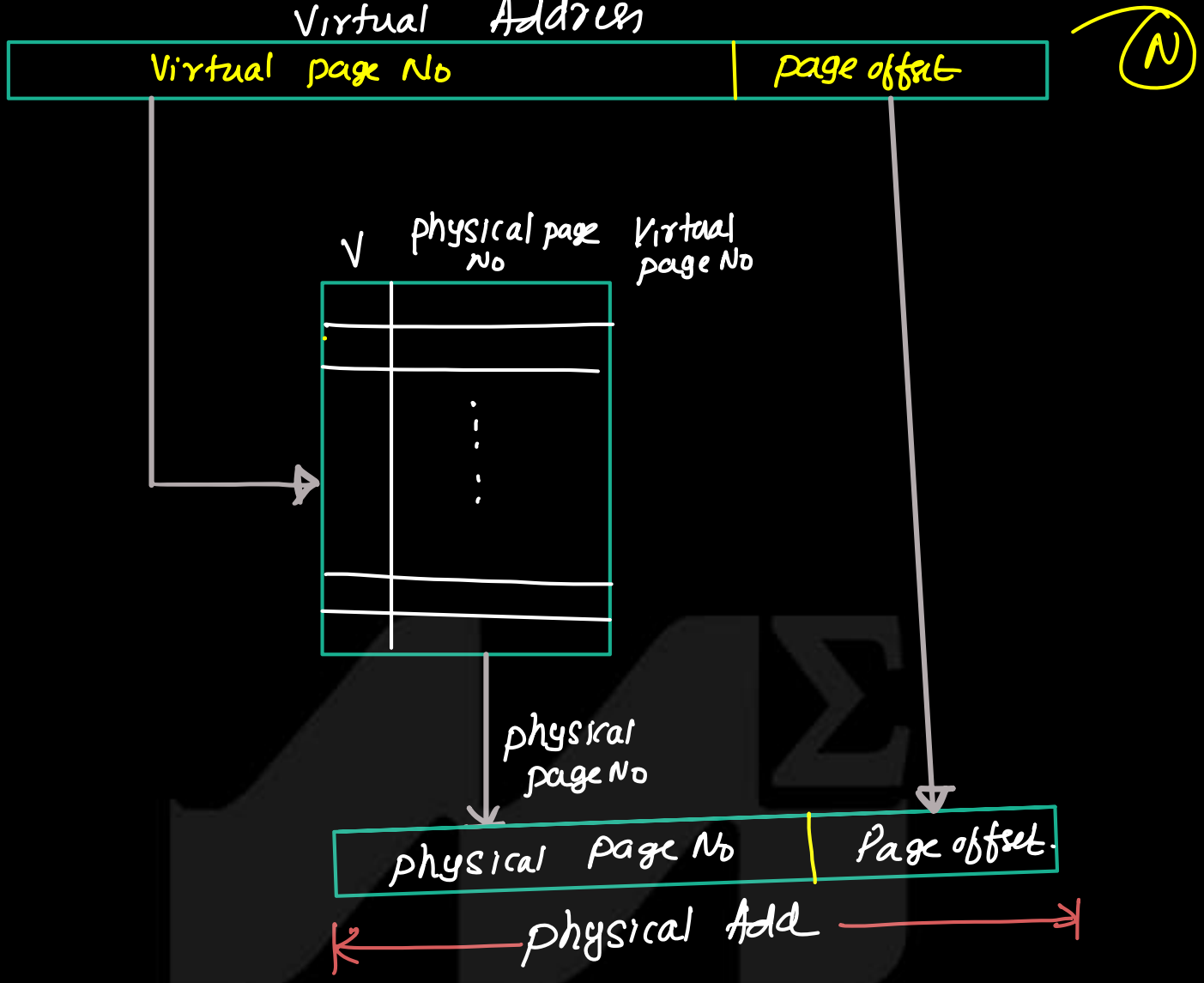
② Extract virtual page number from the virtual Address



③ Find the page table entry for the corresponding virtual page number

④ Processor reads the page table entry

⑤ If valid bit for the page table entry is 1, then processor merges physical page number with page offset to create physical Address.



→ During the program execution, cpu generates virtual address. From the virtual address, the virtual page no: is extracted.

— processor access the page table to find the page table entry. If the physical page no is present it is combined with page offset to generate physical Address.

— once physical address is formed, the cpu access the main memory for the required data.

— Thus to perform a single memory operation Fnt translate virtual address to physical Add' by accessing pagetable and access the main memory for the data. this result two memory access, which increase the memory access time.

→ This problem can be solved using translation look aside buffer (TLB).

Consider a virtual memory system with the following specifications:

Virtual Address Space: 64 KB

Physical Memory (RAM): 32 KB

Page Size: 4 KB

Sample entries of the current Page Table is as follows .

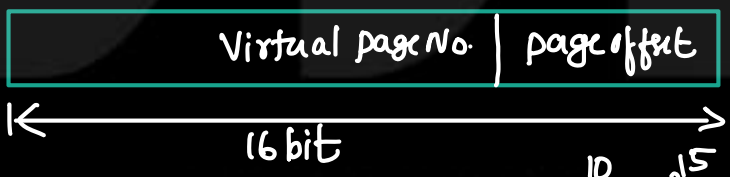
Page Number	Frame Number	Valid/Invalid Bit
0	2	1
1	-	0
6	3	1
10	5	1
...	-..	0

Determine the number of bits in the Virtual Address and the Physical Address. Find the size of the Page Table. Translate the Virtual Address 24586 (decimal) into its corresponding Physical

Virtual Address space = 64KB $= 64 \times 2^{10} = 2^6 \times 2^{10} = 2^{16}$

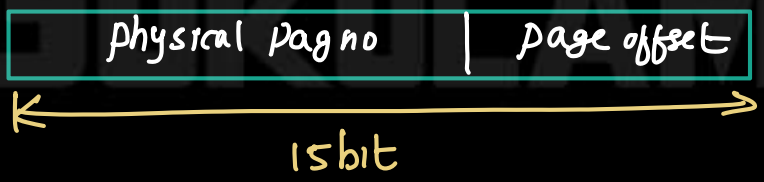
no: of bits to represent Virtual Add $= \log_2 2^{16} = 16$ bit

VA



physical Memory = 32KB = $32 \times 2^{10} = 2^{15}$

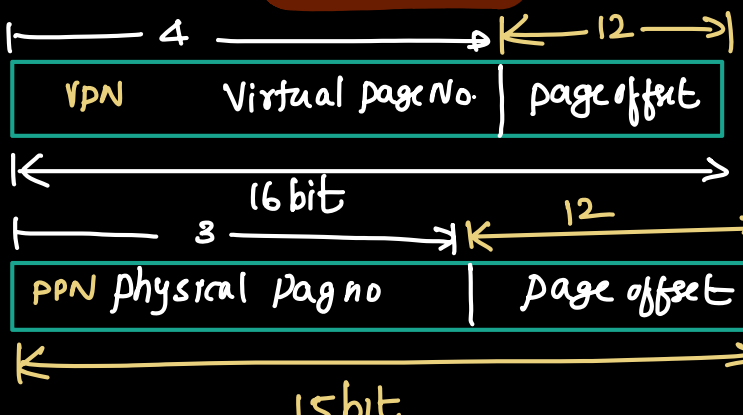
no: of bits to represent physical Add $= \log_2 2^{15} = 15$ bit



Page size = 4KB $= 2^2 \cdot 2^{10} = 2^{12}$

page offset $= \log_2 2^{12} = 12$ bit

Virtual Add



VPN = 16 - 12 = 4 bit

No: of pages in Virtual Memory = $\frac{\text{Virtual Memory size}}{\text{page size}}$

$= \frac{64KB}{4KB} = 16$

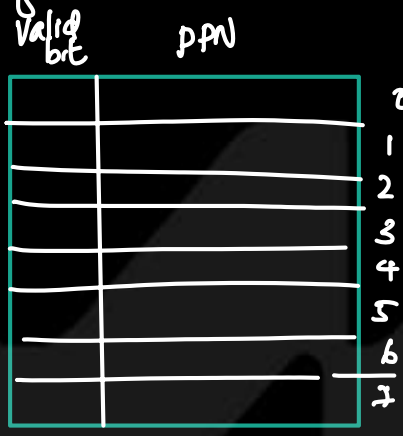
No: of pages in physical memory = $\frac{\text{physical Mem size}}{\text{page size}}$

$= \frac{32KB}{4KB} = 8$

Page table →

no: of Rows → no: of pages in Virtual Memory = 16

no: of columns → 2 (valid bit, physical page No.



dimension of page table = 16 Row X 2 column

in one Row → valid bit + PPN bit = 1 + 3 ⇒ 4bit

Size of page table = 16 x 4 = 64bit

Given Virtual Address 24586 (decimal)

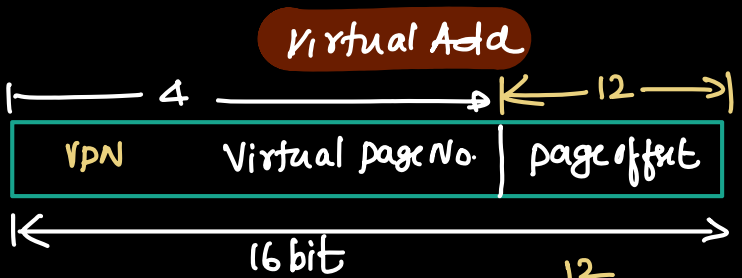
Virtual Add is (16bit)

— find Hex of 24586

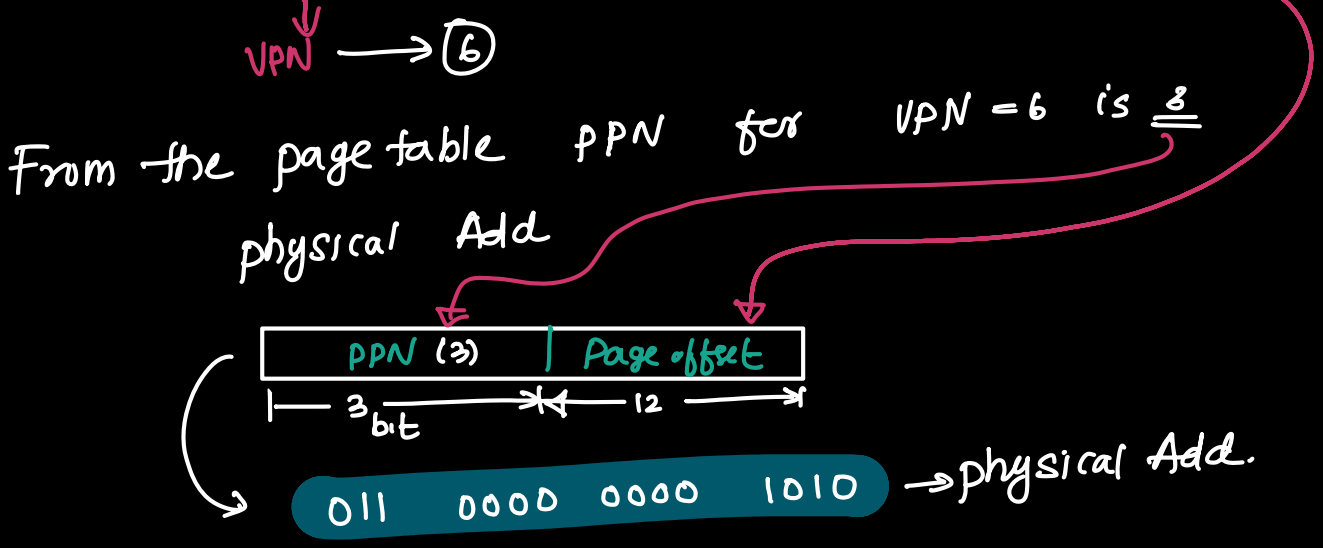
→ 24586 ⟶ $(600A)_H$

16 | 24586
16 | 1536 A
16 | 96 0
6 0

600A ⟶ 0110 0000 0000 1010

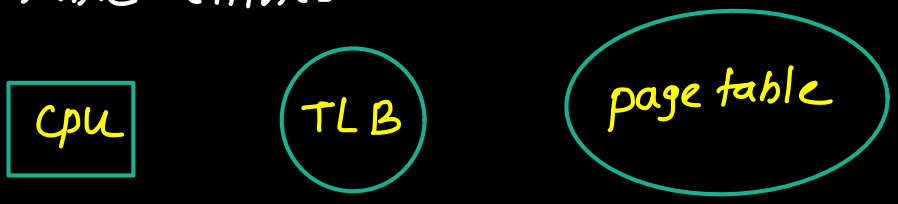


600A ⟶ 0110 | 0000 0000 1010 (12) — page offset



The transition look aside Buffer (Note)

— TLB is a small, fast cache that store recently used page table entries



→ When CPU generate virtual address, the virtual page number (VPN) is first searched in the TLB.

Case I. (VPN found in TLB)

→ If VPN is found in the TLB, the corresponding physical page number (PPN) is obtained directly. This is refered to the TLB hit. Then PPN is combined with the page offset to form the physical address.

→ Then main memory is accessed directly without accessing the page table.

Case-2 (VPN is not in TLB)

— If VPN is not found in the TLB, then page table in the main memory is accessed. and check PPN corresponding VPN is available in the page table.

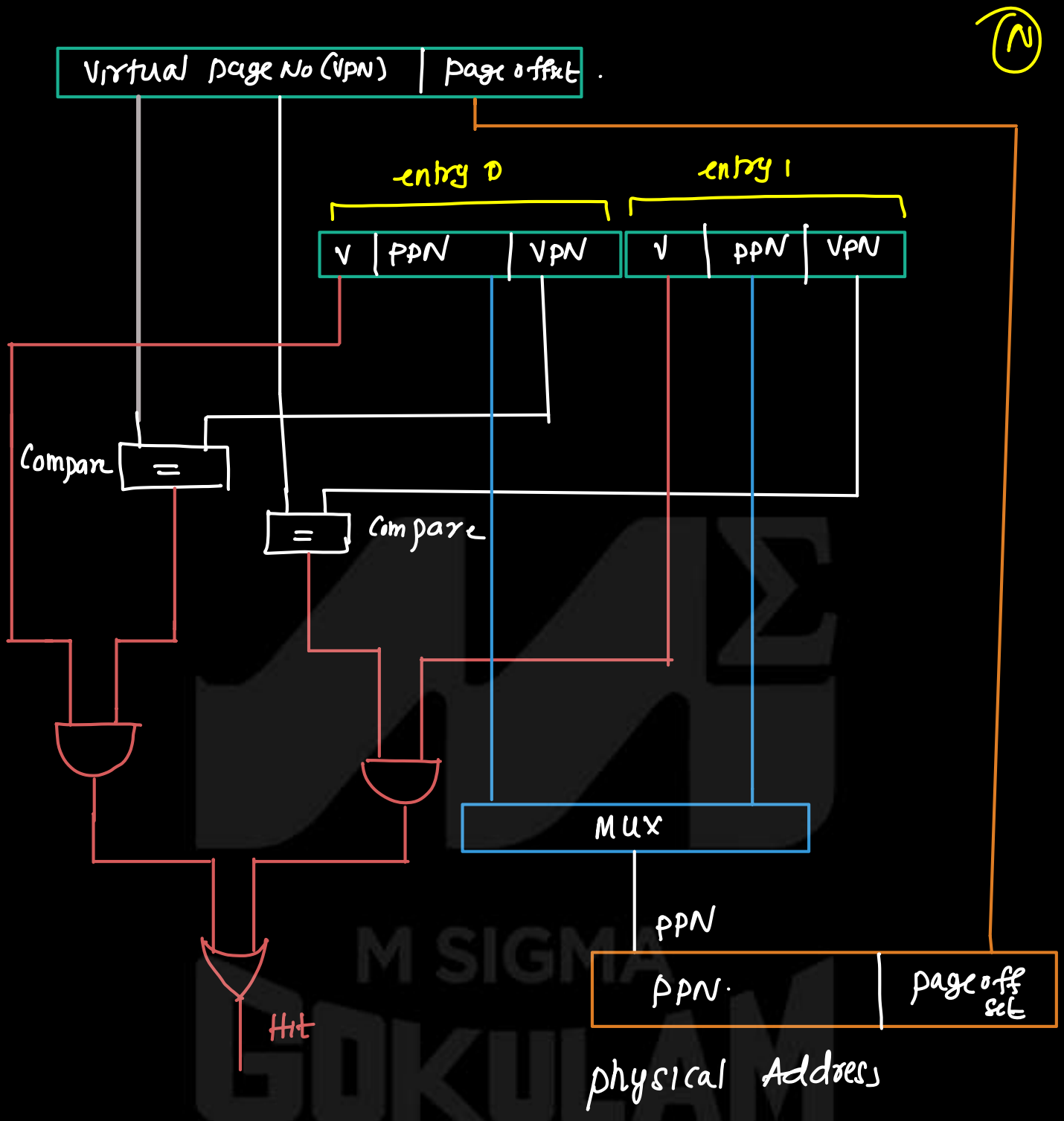
→ If VPN is available, then TLB is updated with new entry

Address translation using TLB

- Consider a TLB with two entries.
  - each TLB entry consists of
    1. Valid bit
    2. physical page No (PPN)
    3. virtual page No (VPN).

- Virtual Address

- TLB receives VPN from Virtual Address.
  - → TLB Compare VPN with VPN of each entry
  - → If VPN from address and VPN of entries are matched, then check for the valid bit.
  - → if valid bit = 1, then TLB contains PPN (TLB hit)
- Concatinate physical page no with page offset.


Virtual page No (VPN)
page offset
entry 0
entry 1
V
PPN
VPN
V
PPN
VPN
Compare
=
=
Compare
MUX
PPN
PPN
page offset
Hit
physical Address

Sample questions

- Explain virtual memory architecture with neat diagrams.
- Describe the process of address translation in detail.
- Explain the working of page tables and TLB in virtual memory.
- Discuss the performance issues of virtual memory and how TLB helps reduce them.
- With an example, explain how a program larger than RAM is executed using virtual memory.

Memory Protection (Note)

Memory protection using virtual memory

Virtual memory is used not only to provide a large, fast, and inexpensive memory system, but also to provide memory protection.

Modern computer systems support multiprogramming, where multiple programs or processes run simultaneously.

Several programs may be present in physical memory at the same time.

In a well-designed system, one program should not be able to access or modify the memory of another program.

Preventing unauthorized access to another program's memory is called memory protection.

Virtual memory provides memory protection by giving each program its own virtual address space.

The virtual address space of a program is independent of other programs and gives the illusion of exclusive access to memory.

Although multiple programs share the same physical memory, each program can access only the physical pages mapped in its own page table.

During execution, programs generate virtual addresses, which are translated into physical addresses using the page table.

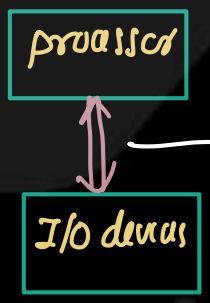
If a virtual page is not mapped in the page table, the program cannot access that physical memory location.

Hence, a program cannot accidentally or maliciously access another program's memory because those pages are not present in its page table.

I/O structure of computer system (Note)

- A computer based system mainly consists of 3 components
  1. Processor (CPU)
  2. Memory
  3. Input / output (I/O) devices.

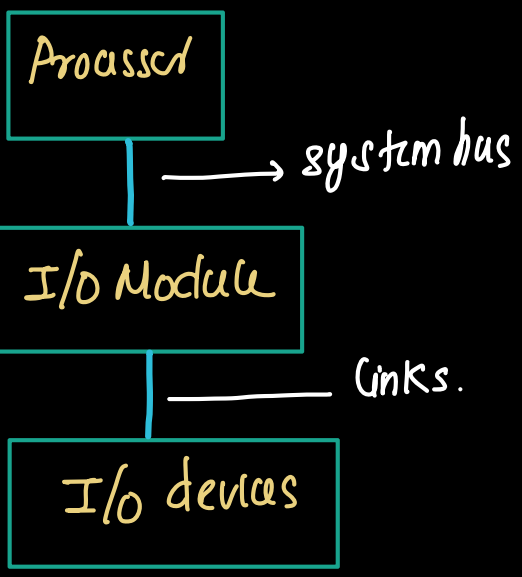
- Processor fetch the instruction from the memory and execute them. During the execution processor need to interract with external devices such as keyboard, mouse.
- To communicate with these device, the processor must access them through an interface
- One possible method is to connect processor and I/O devices directly using system bus



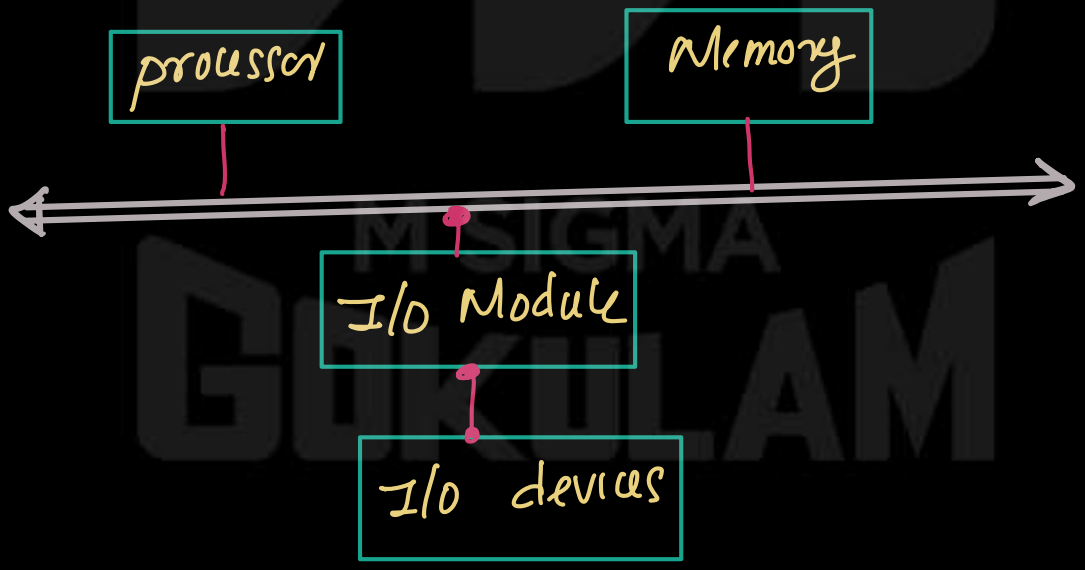
- But direct communication b/w the processor and I/O devices is not efficient because of the following reasons.
  1. Processor operates at very high speed and most of the I/O devices are slow. So there is a speed difference b/w processor and I/O devices.
  2. Because of the speed mismatch, processor would have to wait for the slower devices, which reduce the system efficiency.

Therefore direct connection b/w processor and I/O devices using system bus is not practical.

- To solve this problem, An I/O Module is used. This I/O module act as interface b/w processor and I/O devices



- Processor is connected to the I/o module through the system bus
- The I/o module is connected to the I/o devices through link
- external devices connected to the I/o modules are called perpheral devices.

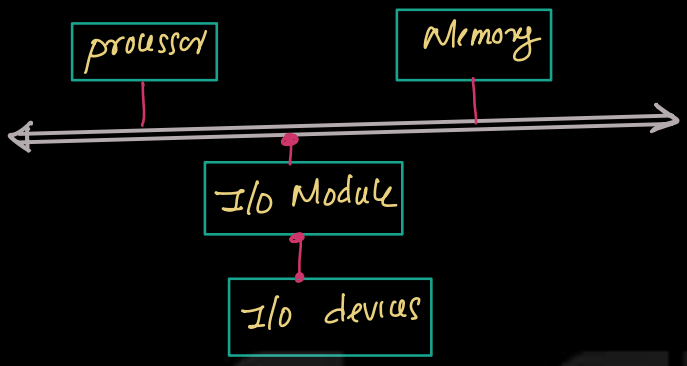


I/O operations

- An I/o operation refers to the ① process of transferring data b/w computer s/m and external devices such as keyboard, printer.
  ② process of transferring data b/w memory and I/o devices

External devices (Note)

Basic computer system consists of processor, memory, sim bus I/O Modules, I/O devices.

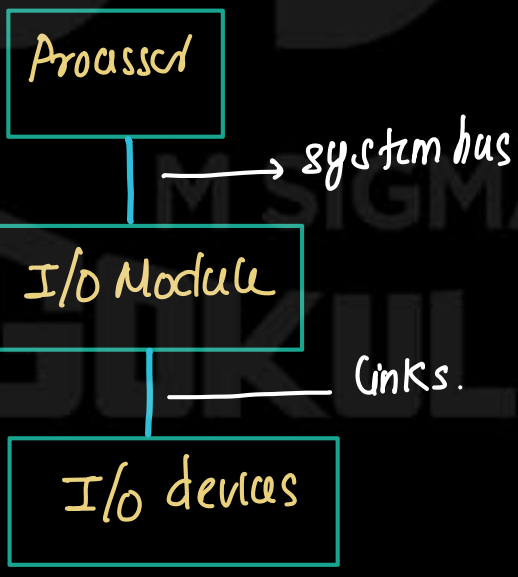


External devices

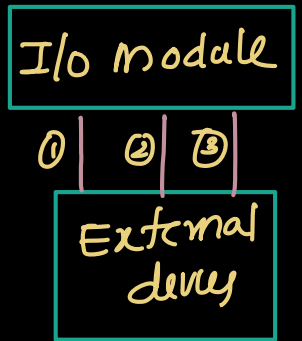
- external devices are the hardware components that allow a computer systems to communicate with the out side environment.
- these devices allow the computer s/m to receive the data, send result and interract with users.
- external devices are not directly connected to the processor because processor operates at a very high speed while most general devices operates slowly.

→ therefore external devices are connected through I/O modules.

- The processor communicates with the I/O module through the system bus and the I/O module communicate with the external devices through a device interface (or) links.

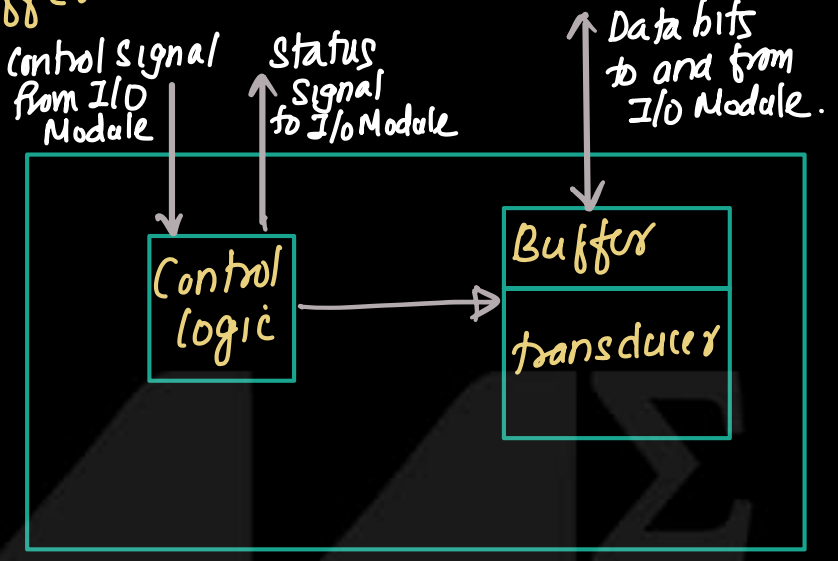


- the connection btw external devices and I/O modules uses three types of signal
  1. Control signal
  2. Data signal
  3. Status signal



Structure of External device

- External device consists of 3 main components
  1. Control logic
  2. transducer
  3. Buffer



Eg: processor needs to read data from I/O device

- The processor send command to the I/O module through the system bus. this command specifies that the processor wants input data from the device
- After receiving command I/O module send a control signal to the I/O devices
- Control logic inside the external device receive the control signal from the I/O module.
- Control logic control the operation of the device according to the instructions received from the I/O module
- → transducer inside the external device continously monitor the physical environment and convert this physical phenomena to electrical signal for input operation.

- → Control logic inside the external device continuously monitor the signal from the transducer

→ when control logic detect a valid signal, then control logic convert this signal into bit pattern

— This bit patterns are temporarily stored in the buffer attached to the transducer.

— external device send a status signal to the I/O module indicating that data is available

— Then buffered data is transferred to the I/O module through data lines.

(1) Control logic

— Control logic inside the device manages the operation of the device

— works according to the instructions from the I/O module.

(2) Transducer.

Tranducer inside the device convert electrical signal to other form of energy and vice versa

(3) Buffer

Buffer temporarily store data during transfer btw external device and I/O module.

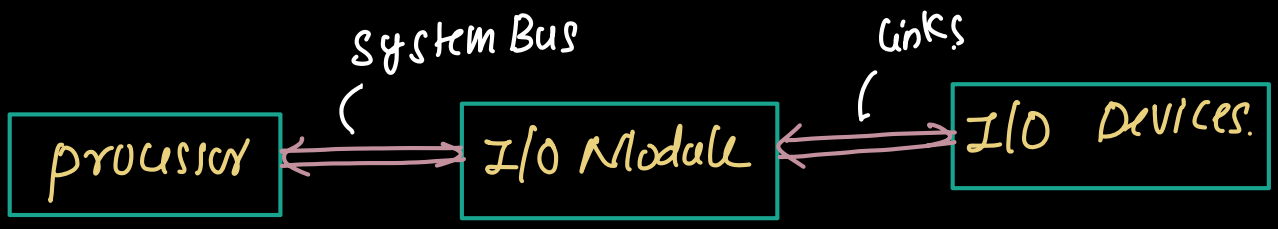
Keyboard input Process

- when key is pressed, it generates an electronic signal
- the transducer in the keyboard convert this signal into electrical signal
- control logic continuously monitor the signal at the transducer. If control signal detect a valid signal. then this signal is translated into bit pattern.
- these bit pattern send to the I/O Module.

I/O Module (Note)

→ I/o module act as interface between processor and external devices such as keyboard, printer etc....

— Since external devices operates much slower than the processor, I/o module manages communication and data transfer btw the processor and these I/o devices.

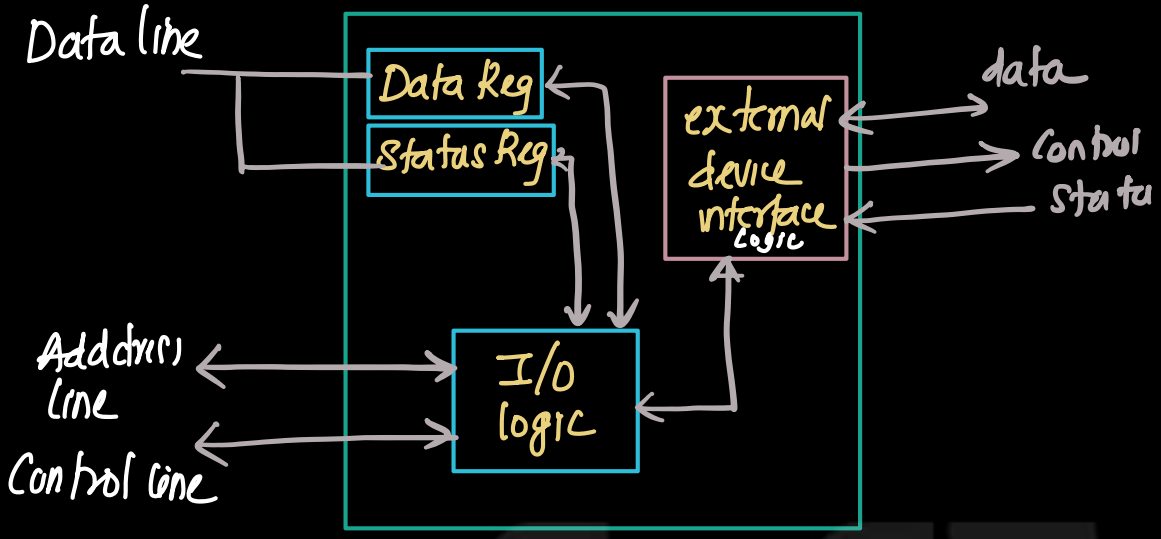


— The processor communicates with I/o module through the system bus (Add Bus, data Bus, Control Bus).

→ I/o module communicate with external device through external device interface.

Structure of I/o Module.

— I/O Module consists of following components that manage the communication btw processor and external devices.



① External device interface logic

- this manages the communication btw I/o module and external device.
- It handles 3 types of signal
  1. data signal:
  2. Control signal
  3. status signal.

② Data Register

- Data Registers are temporarily store the data that is being transfer between the external device and processor.
- Eg: when key is pressed on keyboard, the generated bit pattern stored in the data register.

② status Register

→ this register store the information about of the device.

→ eg: If key is pressed, then bit pattern stored in the data register, and update the content

of status register (device is ready). (N)

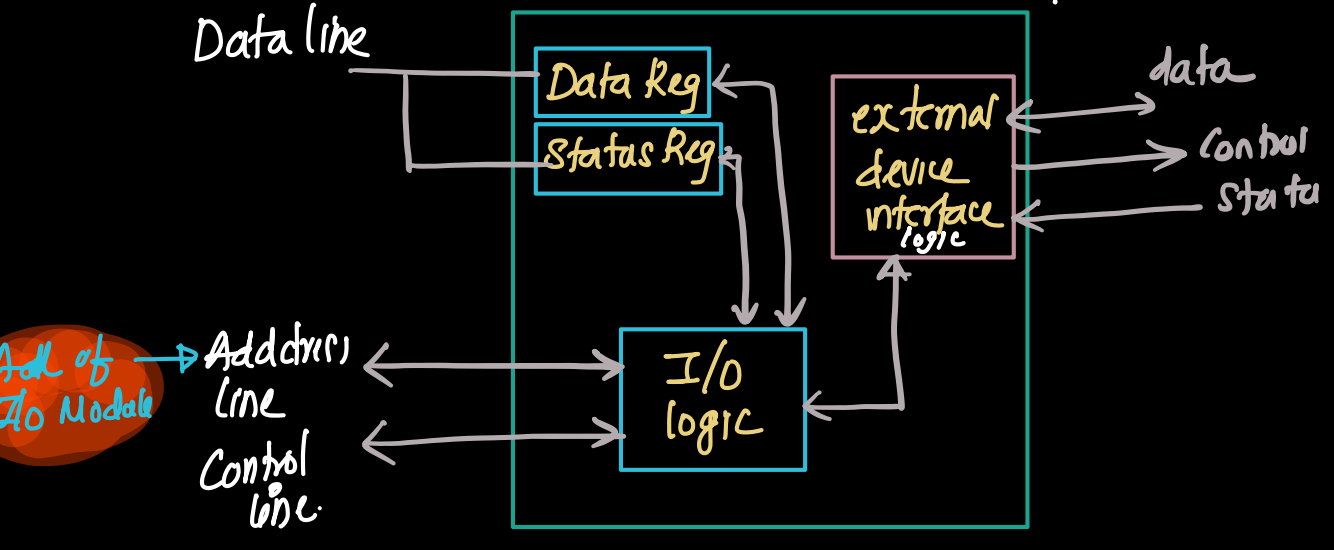
- information includes
  1. ready – device is ready to transfer
  2. Busy – device is busy

(3) I/O logic

- I/O logic control the operation of I/O module.
- → it perform the following functions
  1. control the data transfer b/w the processor and external device
  2. Co-ordinate communication b/w external device interface (Logic) and Registers

Data transfer from I/O device to the processor.

- Consider an I/O device is connected to the processor through I/O module
- Eg: Reading a character from the key board
- processor place the Address of I/O module on the address line of the system bus
- This select specific I/O module and device that processor wants to access



— processor activate control signal on the on the control bus

eg: To read data from the device, processor activate I/O Read signal

To send data to the device, processor activate I/o write signal

these signal specify the type of operation the processor wants the I/o module to perform.

(processor send activate IO Read signal to the control line of system bus

— I/o logic inside the I/o module decode the control signal. From this it understand that the processor is requesting a read operation.

— After decoding control signal, I/o module sends the appropriate control signal to the external device through the external device interface logic. This tells the device to provide the required data.

— the device sends the data to external device interface Logic.

— external device interface logic receives the data and temporarily store in the data Register and updates the status register, indicating that input data is available (Ready bit of status register become 1).

Ready bit = 0 (no data available).

Ready bit = 1 (data is available)

— After sending read command, processor read the status register of I/o module. status register contain information such as

- device ready
- device busy.

— processor check whether the device is ready to transmit when the device become ready, I/O module place the data in data lines of system bus and processor read the data

Sample Questions

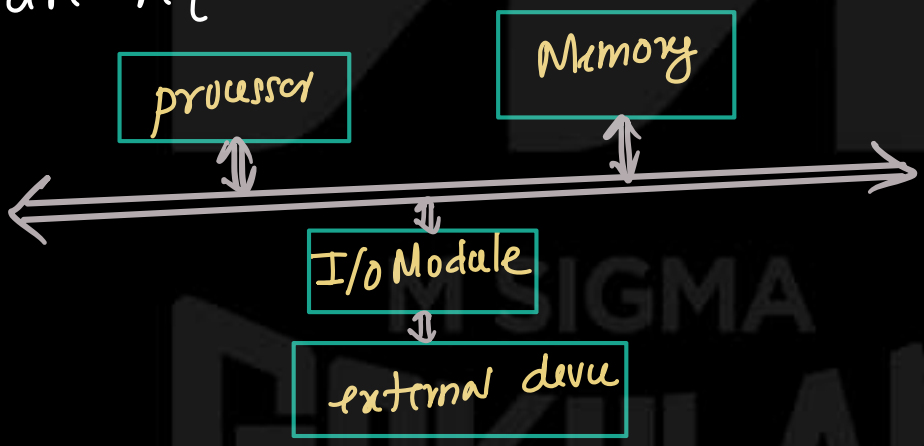
1. Functions of I/O module
2. structure of I/O Module
3. How data transfer btw an external device and processor

Methods to perform I/O operation (Note)

- I/O operations refers to the process of transferring data between the computer (processor & Memory) and external device
- I/O module is used as interface to manage communication b/w processor and external devices.
- During I/O operation processor issues commands to the I/O module, which communicates with external device and transfer the data between the device and processor.

Need of special I/O techniques

- external device operate much slower than the processor. Because of this speed difference, special techniques are required to coordinate the data transfer



- Main techniques used for the I/O operations are
  1. Programmed I/O
  2. Interrupt driven I/O
  3. Direct Memory Access (DMA)

Programmed I/O

- Programmed I/o is a technique in which the processor controlls all I/o operations directly.
- When processor execute an instruction related to input/output, it sends command to the I/o module.
- I/o module performs the requested operation and update status register to indicate the condition of the device
- → therefore processor continuously check the status register of the I/o module to determine operation has finished. (Device is ready)
- This repeated checking of device status is called polling.
- Thus in programmed I/o, the processor remains actively involved in the entire I/o operation.

Eg: We need to read a block of data from I/o device using programmed I/o.

Step 1. processor issue an read command to the I/o module.

- This command instruct the I/o module to obtain data from the external device.
- during this time I/o module send the control signal to external device and external device send the data to external device interface logic.

→ then data is temporarily stored in data register and update the device status in status register.

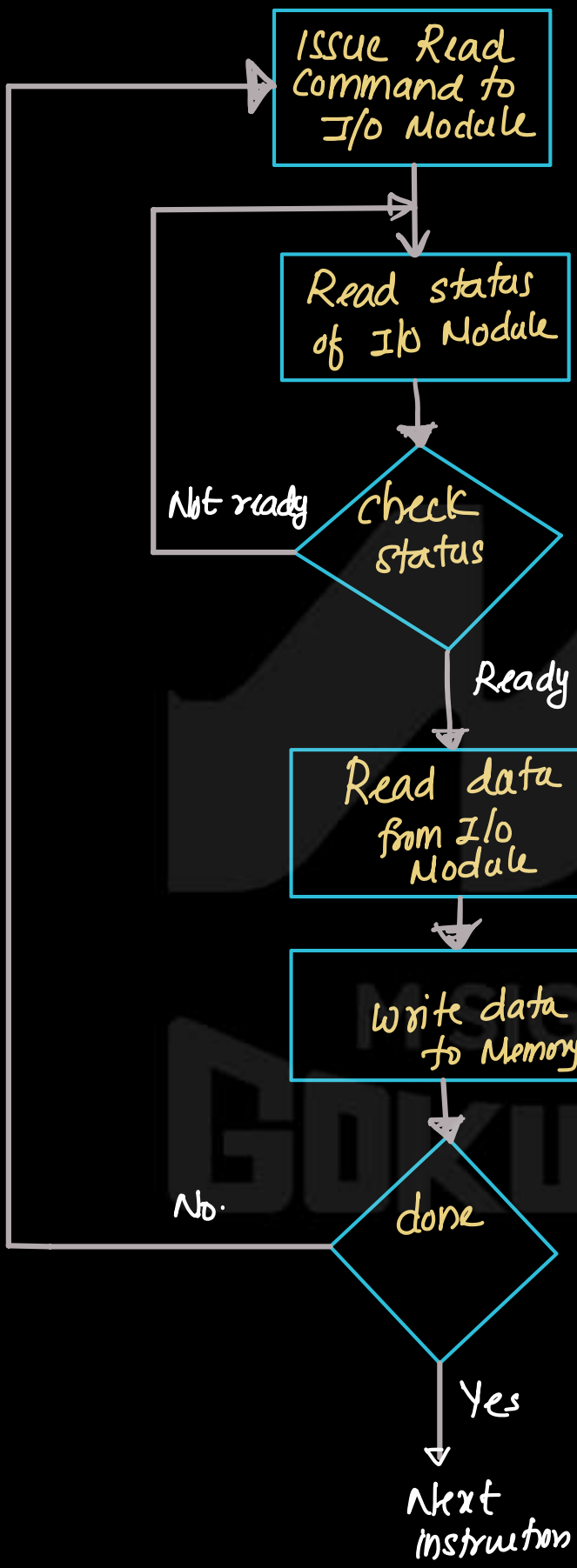
Step-2.: After issuing the read command, processor reads status register of the I/O module.

Status Register contains information such as
① device ready
② device busy.

Step 4: Read data from I/O module
– when device become ready, processor reads the data from I/O module through data Bus.
– thus data transfer to the processor.

Step 5: Write data into memory.
After receving the data, the processor store the data into main memory.

Step6 : The processor check whether the entire block of data is transfered or not.
If entire block of data transfered, then processor proceed to execute next instruction
If more data remains to transfered, processor return to the first step and repeat the prouss.


Issue Read Command to I/O Module
Read status of I/O Module
Check status
Not ready
Ready
Read data from I/O Module
Write data to Memory
done
No
Yes
Next instruction

Disadvantage

- Processor must continuously check the device status.
- During that time processor cannot perform other useful tasks. As a result processor time is wasted, making programmed I/O an inefficient technique for data transfer.

Addressing of Memory and I/O devices (Note).

- In a computer system, the processor, main memory and I/O module communicate through a common system bus.
- The system bus consists
  1. Address bus
  2. data bus
  3. Control bus.
- When the processor needs to access the memory, it places the address of memory location on the address bus. memory unit recognizes this address and transfer the required data through the data bus.

→ Similarly when the processor needs to communicate with an I/O device. processor places address of I/O devices / add of I/O module register on the address bus. The corresponding I/O module recognize the address and exchange the data through system bus.

- Since memory, processor, I/O devices share the same system bus, processor must able to distinguish btw memory address and I/O address. two addressing methods are used for this purpose.
  1. Memory mapped I/O
  2. Isolated I/O.

Memory Mapped I/O.

- In memory mapped I/O, a single Address space is used for both memory location and I/O devices (I/O Module Register).
- Processor treat the status and data register of I/O module as memory locations.
- Therefore processor use the same instruction to access both memory and I/O devices.
- Eg: If the system bas have the 10 Address lines.

  Total No: of Address = 2^{10} = 1024

  - First 512 Address are used to address the memory location.
  - Remaining 512 Address are reserved for I/O module Registers / I/O devices

    eg: 516 → assigned for data Register
    517 → assigned for status Register.

→ Eg: - A keyboard is connected the computer s/m. and processor need to read data from keyboard.

- two memory Add are reserved for the keyboard
  517 — Keyboard status Register
  516 → Keyboard data Register.

program.

① processor load the value 1 into the accumulator

load Ae, 1

② processor store the value 1 into address 517 this initiates the keyboard read operation

load AC, SIZ

③ processor check the status Register (SIZ) to determine whether the data is ready

load AC, SIZ

④ processor repeatedly check the status Register until the ready bit becomes 1.

```
        load  AC, 1
        store AC, SIZ
L1      load  AC, SIZ
        Branch if sign = 0 ; L1
```

⑤ Read the input data

If ready bit become 1, then processor reads the data from data Register

```
        load  AC, 1
        store AC, SIZ
L1      load  AC, SIZ
        Branch if sign = 0 ; L1
        load  AR, SIB
```

Advantages

→ same instruction can be used to access the memory and IO devices., which allows more flexible and efficient programming.

An embedded system designer is developing a specialized industrial controller using a RISC-V processor. The system needs to interface with a high-speed sensor that provides 16-bit data samples. The designer is considering using Programmed I/O to manage data transfers. Explain the step-by-step mechanism the processor will follow to read a single data word from the sensor

Explain Programmed I/O.

Isolated I/O (Note)

- In isolated I/O, the address space used for I/O devices is separate from memory address space. Therefore memory and I/O devices have independent address ranges.
- For eg: , with 10 address line, the system may support
  1024 memory Address
  1024 I/O addresses.
- In this technique, processor communicates with I/O devices using spacial I/O instructions such as IN and OUT
  IN – to read data from an I/O device,
  OUT – to send data to an I/O devices

Interrupt driven I/O (Note)

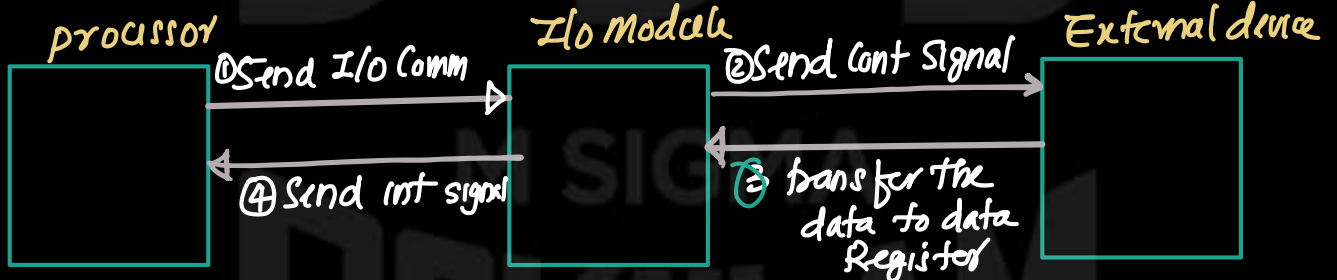
→ In programmed I/O, processor must continuously check the status of the I/O module to determine whether the device is ready to transfer the data.

— thus processor cant do other usefull work. which result in a wasting processor time and degrading system performance.

— To overcome this, interrup driven I/O is used.

— In Interrupt driven I/O, processor issue an I/O command to the I/O module. and processor continue the execution of other instruction.

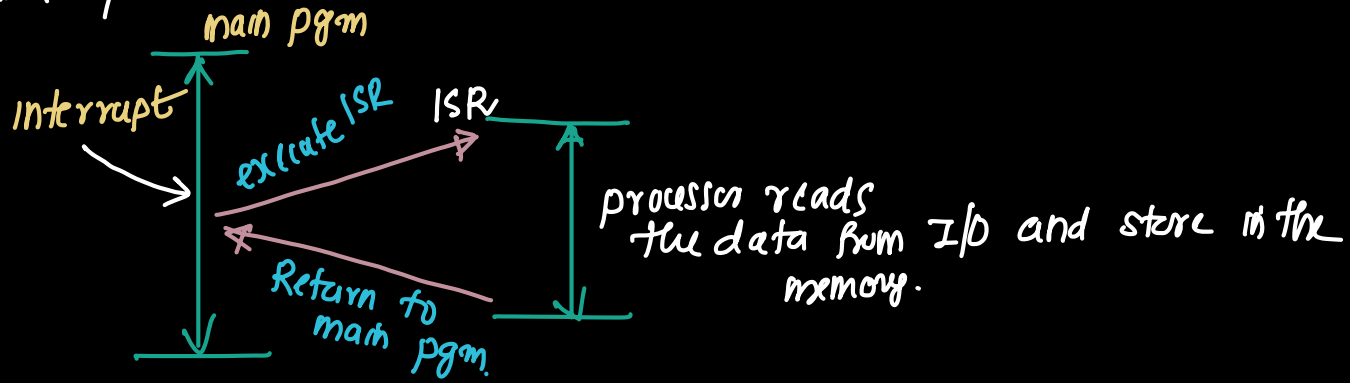
→ On receiving I/O command, I/O module read the data from corresponding peripheral device and store the data in data register of I/O module.



→ when data is available in the I/O module. I/O module send an interrupt signal to the processor.

→ processor complete the execution of current instruction and process the interrupt.

→



→ Processor execute the ISR. During the ISR execution processor reads the data from the I/O device and store the data in memory. once ISR is completed processor resumes the main pgm.

Operation of interrupt driven I/O (Note)

- Processor send a read command to the I/O module
- The I/O module read data from associated peripheral device
- The data stored in the data Register of the I/O module.
- when the data becomes available, the I/O module sends an interrupt signal to the processor indicating that it require service (Interrupt Request)
- processor complete the execution of current instruction before responding to the interrupt. (Completion of current instruction)
- processor checks for interrupts, determine interrupt has occured, and send acknowledgment signal to the device. This acknowledgment allows the device to remove its interrupt signal. (Interrupt Acknowledgment)
- Processor saves information required to resume the main program.
  - Address of next instruction (Content of PC) and
  - status information (Content of PSW)

  then values are pushed to the stack. (Saving processor state)

→ processor transfer control to execute interrupt service routine. The processor loads the Add of ISR to the PC (loading interrupt Service Routine)

→ During ISR execution processor read the status Register of the I/O module. Depending on the operation

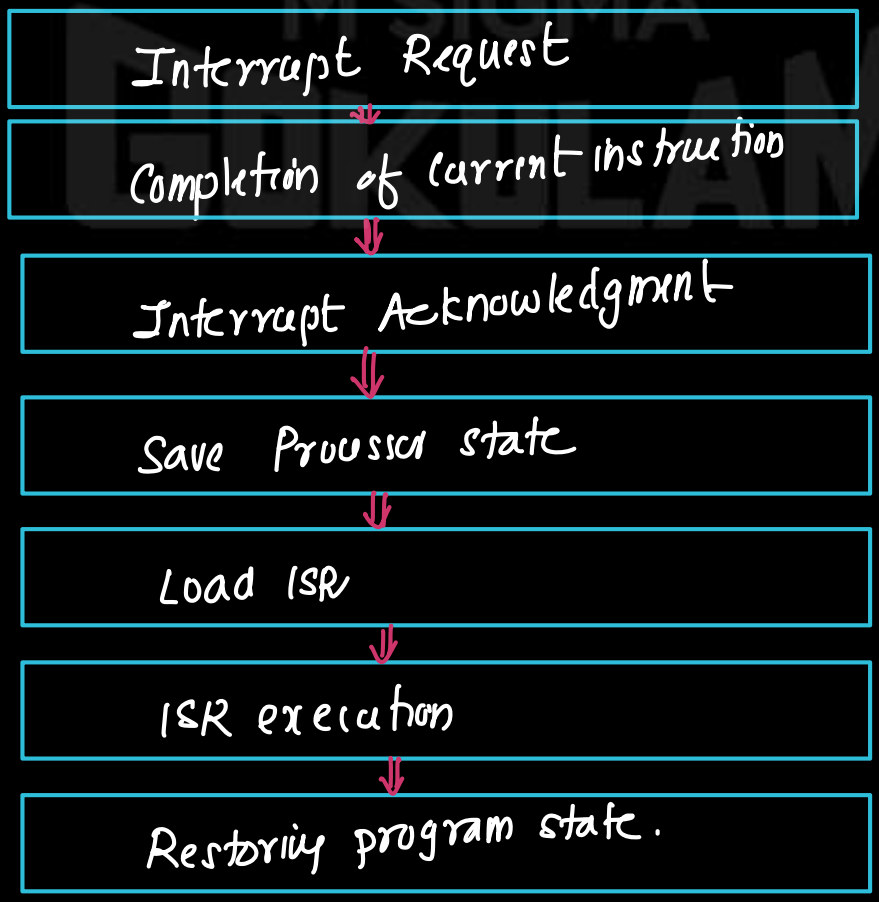
① Input operation
The processor read the data from data Register of the I/O module

② Output operation
processor write the data into data Register (ISR execution)

- After all required actions are completed, content of PC and PSW are restored and processor returns to the main program and continue the execution from where it was stopped. (Restore program status)

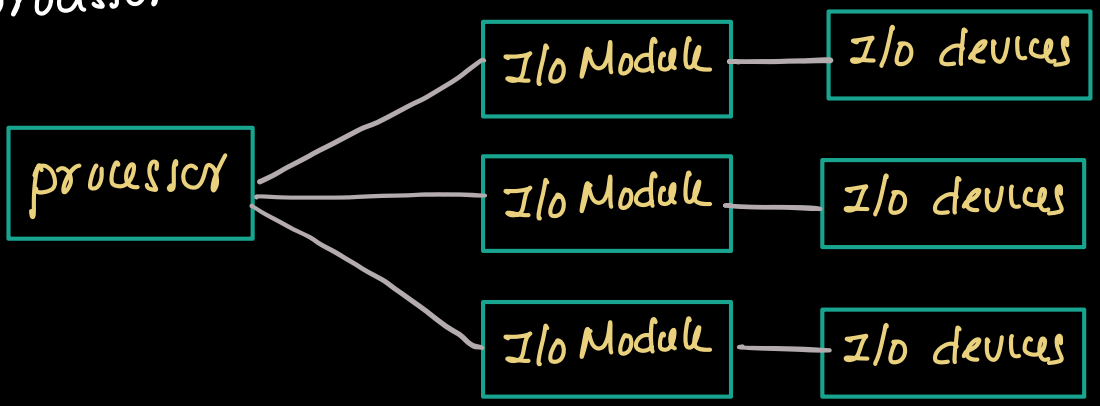
Interrupt processing steps.

1. Interrupt Request
2. Completion of current instruction
3. Interrupt Acknowledgment
4. Save Processor state
5. Load ISR
6. ISR execution
7. Restoring program state.



Design issues in Interrupt driven I/O (Note)

— In a computer system, multiple I/o devices are connected to the processor



→ when interrupt driven I/o is used, there device may generate interrupt request to the processor.

Design issues

— When interrupt occur, processor must determine which I/o device / I/o module generated the request.

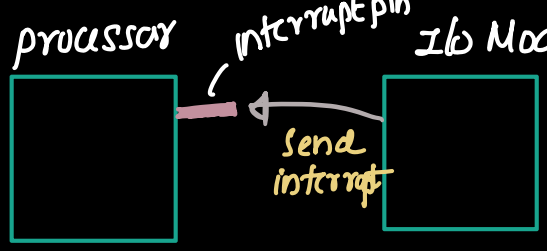
— If multiple interrupt occurs at same time, processor must decide which interrupt should be serviced first.

The first issue (identify the device which request interrupt) can be solved using following techniques.

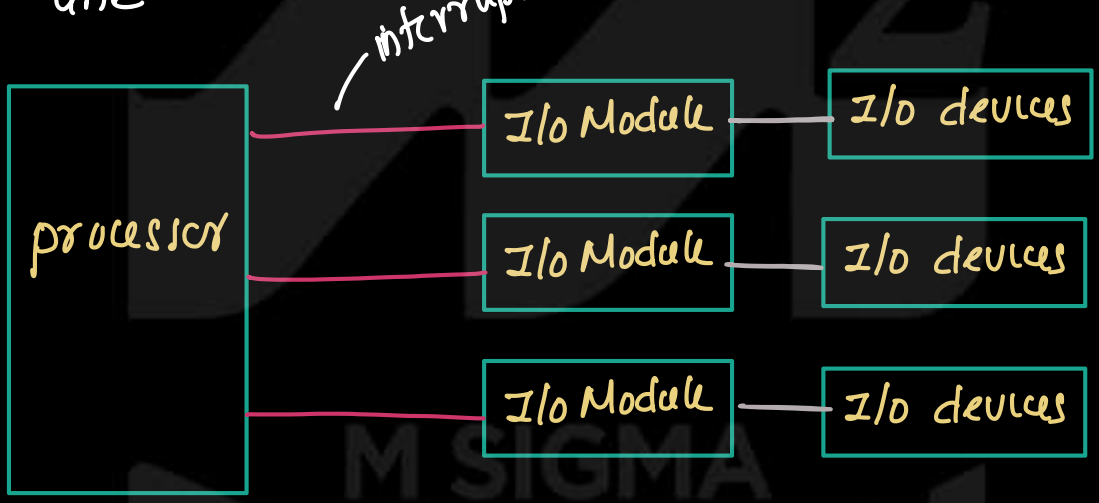
1. Multiple interrupt line
2. Software polling
3. Daisy chain.

① Multiple interrup line.

- normaly processor have dedicated pin to allow an I/o device to send an interrupt signal



- To identify the device who generated interrupt, multiple interrupt lines are provided
- Each I/o module is connected to seperate interrupt line



→ when an device / (I/o) module need the service from the processor, it raises its own interrupt line. thus processor can immediately identify which device has generated the interrupt

Disadvantage

① It is impractical to provide the large no: of interrupt lines because the processor pins are limited

② Software Polling

→ When an interrupt occurs, processor execute and interrupt service routine that check each I/O module one by one

→ when processor detect an interrupt, processor executing a polling routine. processor check the status register of each I/O module one by one.

. when the module that generated the interrupt is found . processor branch to that device specific ISR.

disadvantage.

– processor check multiple devices sequentially (one by one) so this technique is time consumming.

③ Daisy chain

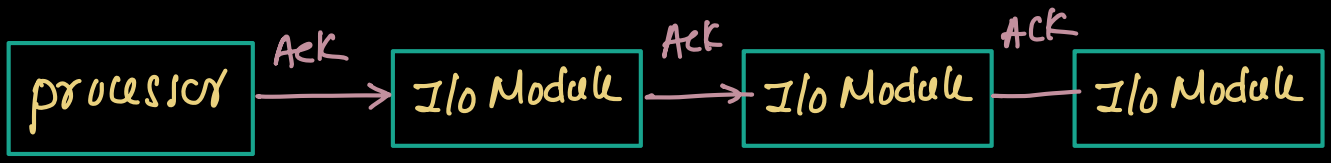
– In this technique, all I/O modules share a common interrupt line connected to the processor.

– Operation

– when an I/O device require service, it sends an interrupt signal through its I/O module to the processor using common interrupt signal

→ once processor detect the interrupt, processor send an acknowlege signal. this signal indicate that processor is ready to determine which device generated the interrupt.

– this ack signal travels through the I/O modules one by one in sequence. this arrangement forms a diasy chain.



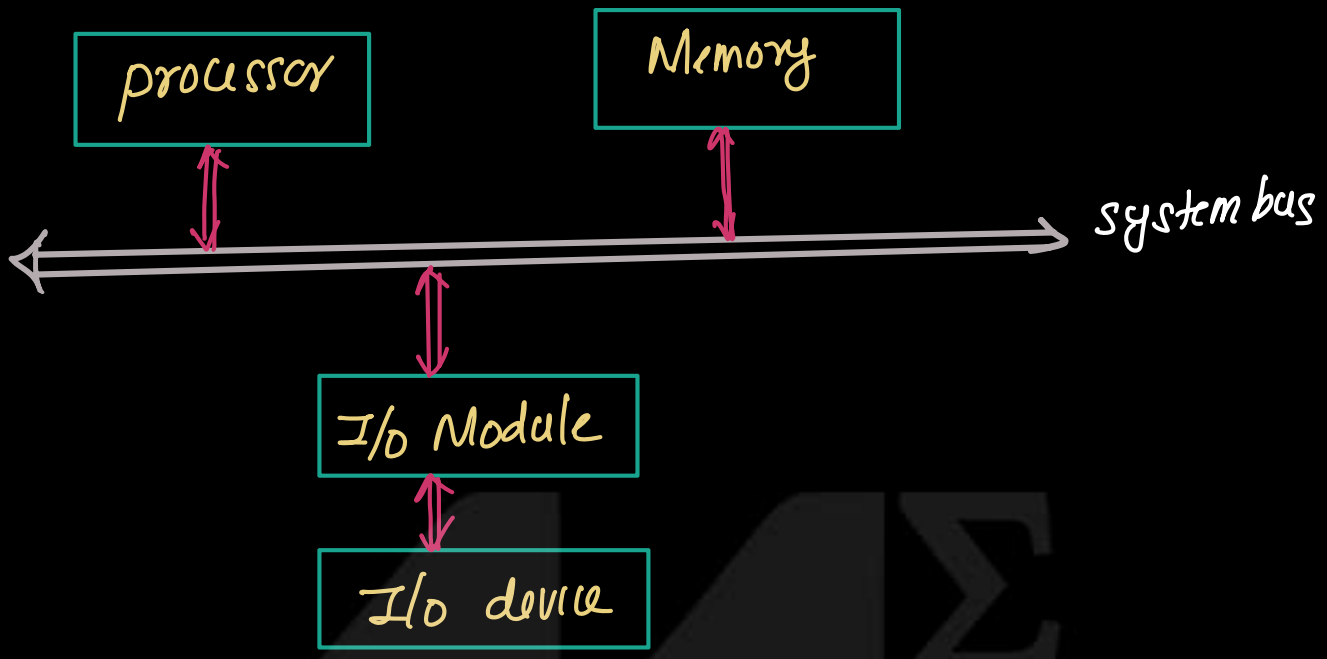
- each module perform the following operation
  1. if it didnot generate the interrupt, it passes ack signal to next I/o module in the chain
  2. if it generated the interrupt, then placing a word on data line. this word is called vector. this represent the address of I/o module

- processor reads the vector from the data bus and ax it to determine which ISR to execute.

SAMPLE QUESTIONS- interrupt driven I/O

- Explain the operation of Interrupt-Driven I/O.
- Describe the sequence of operations in interrupt processing.
- Explain interrupt processing with the help of a diagram.
- Explain the steps performed by the processor when an interrupt occurs.
- Explain the working of Interrupt-Driven I/O from the point of view of the processor and the I/O module.
- What are the design issues in interrupt-driven I/O?
- Explain different methods used for identifying the interrupting device.
- Explain the following interrupt identification techniques:
    - Multiple interrupt lines
    - Software polling
    - Daisy chain

Direct Memory Access (Note)



- Consider an I/o operation in which block of data must be read from the I/o device and store the data in memory.
- In a computer system, processor, memory, I/o device (I/o module) are connected through the system bus.

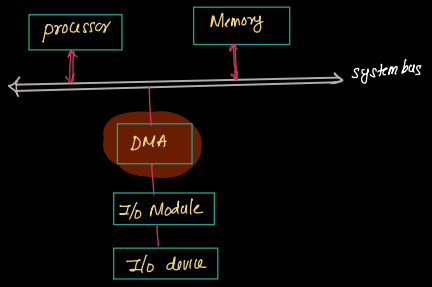
— If programmed I/O is used for this I/O operation, processor is completely dedicated to managing the data transfer. processor must repeatedly check the status of I/O devices and transfer each word of data b/w device and memory. During this time processor cannot perform other useful task. Therefore processor is not used efficiently.

— In interrupt driven I/O, the processor issues an I/O command to I/O module. and processor perform other task. When the device become ready, it generate an interrupt and processor perform the data transfer. Although this method improves processor utilization, processor must still participate in each data transfer b/w I/O module and Memory.

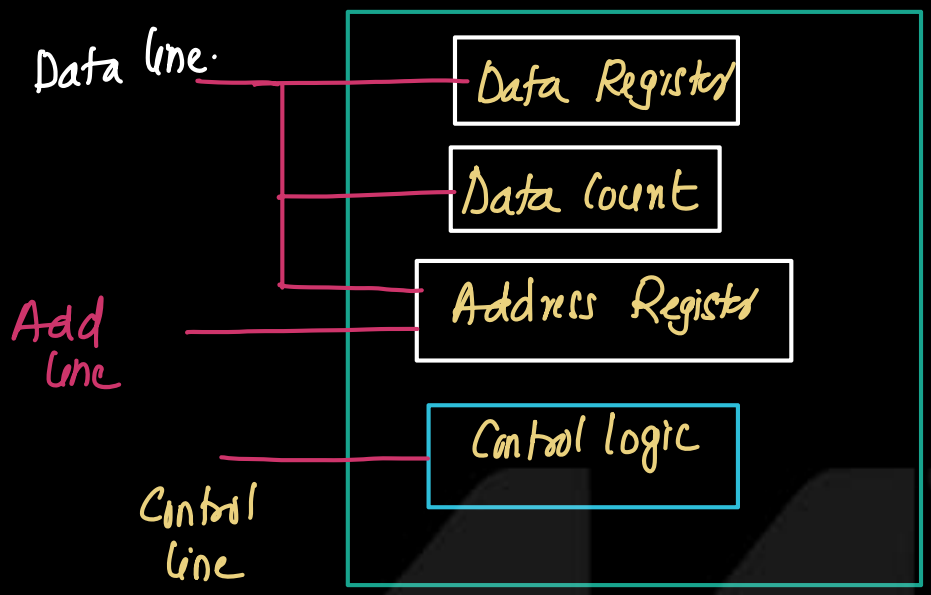
— When a large block of data must be transfered, both programmed I/O and interrupt driven I/O are inefficient. Therefore a technique is required that allows the I/O operation (data transfer b/w I/O device and Memory) to occur without the continuous involvement of the processor. this technique is called Direct Memory access (DMA).

→ DMA allows data to be transfered directly b/w I/O devices and memory without continuous involvement of the processor.

→ In this method special hardware unit called DMA module (DMA Controller) is connected to the system bus.



Block diagram of DMA



DMA function

(N)

- Consider an I/O operation in which block of data must be read from an I/O device and stored in memory.

- Step-1

When processor wants to read a block of data from an I/O device, it issues a command to the DMA module. The processor send following information to the DMA

① Address of I/O devices (I/O) module

the processor send the add of I/O device so that DMA controller knows which device will participate in the data transfer.

(2) The processor specify the operation req to perform (Read / write) using control signal through control lines of processor. this control signal is interprete by control logic inside the DMA

(3) processor send start Add of memory location where the data will be stored. this Add is stored in the Add register of DMA Control.

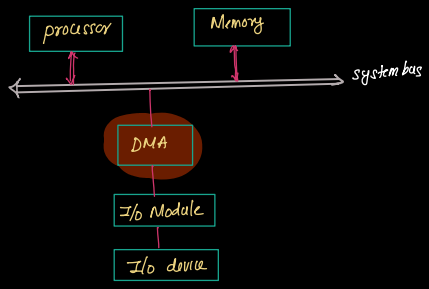
(4) processor send the number of words (data) to be transferred to count Register inside the DMA

Step-2

→ After providing all the information, processor Continue executing other instruction. thus DMA co:ordinate the data transfer operation

Step-3

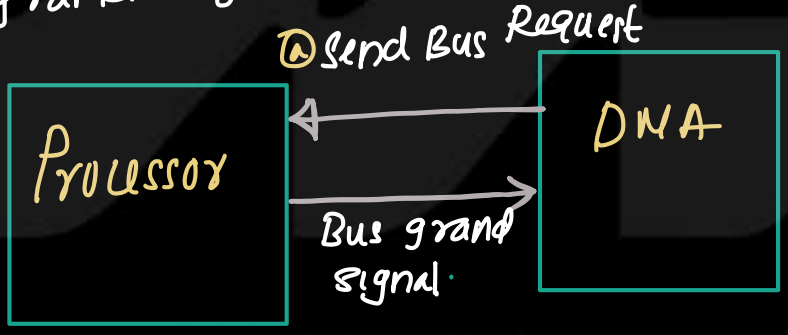
- In a computer system processor and the DMA share the same system bus to access the memory.



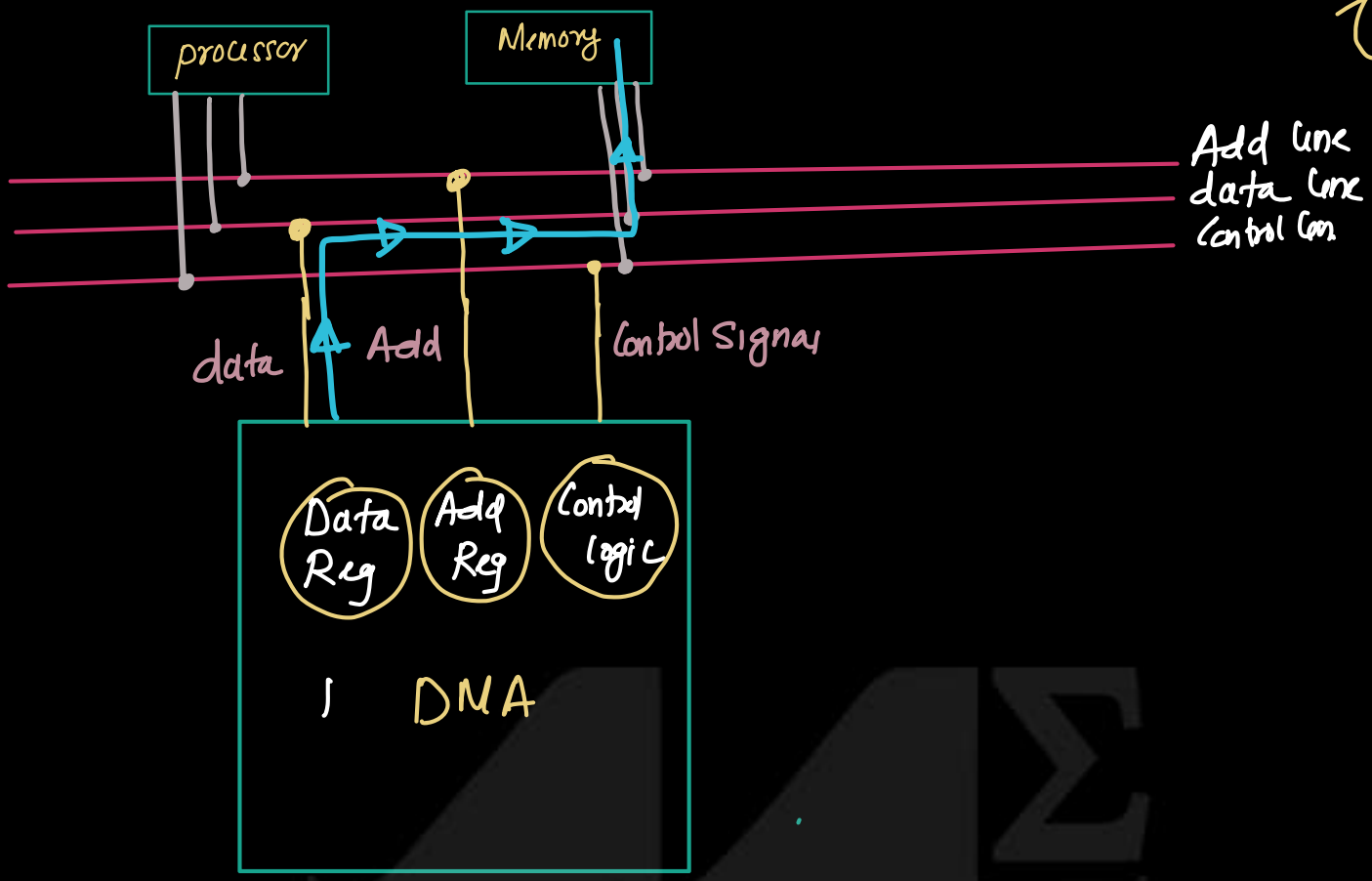
- Since only one component can use the bus a time.

→ To transfer data btw I/o device and memory, DMA send a Bus Request signal to the processor.

· Processor completes the current bus operation and then grand control of system bus to DMA by sending Bus grand signal.



Step-4 - DMA transfer the one block of data directly btw I/o module and memory.



Step 5: After transferring one word, DMA Controller releases the bus, allowing the processor to continue its execution.

this technique is called bus stealing, because the DMA controller steals one bus cycle from the processor to perform the task.

eg: instruction execution includes following step

① Fetch – Fetch instruction
② Decode – Decode instruction
③ Fetch operands –
④ execution → perform the task
⑤ Memory access →

In the Fetch, Fetch operand, memory access stage use s/m bus and decode, execution not using s/m

Fetch	Decode	Fetch operand	Execute	Memory access

let say processor Finish decoding, Next step is Fetch operands. Before starting Fetch operand processor need s/m bus to access the memory. At this moment DMA request has, processor pauses temporarily. DMA transfer one word of the data. The bus is returned to the processor. Then the processor continue fetch operand.

Step-6 – DMA Continues transferring data one word at a time until the data count register become zero. this indicate that entire block has been transferred.

Step-7 — After the transfer is complete, DMA send an interrupt signal to the processor to indicate that DMA operation finished

Summary:

1. processor send information to DMA (send command)
2. processor continue execution of other instruction
3. DMA request system BUS (sending Bus request signal)
4. Processor complete current bus operation and send bus grand signal
5. DMA transfer the data (stealing bus cycle) one word at time
6. DMA complete the data transfer and send interrupt signal to the processor.

Module - 4

Q7: Differentiate between 'Cycle Stealing' and 'Burst Mode' in DMA.

Answer:

Aspect	Cycle Stealing	Burst Mode
Bus Control	DMA steals one bus cycle at a time	DMA takes full control of the bus for the entire transfer
CPU Impact	CPU is briefly paused for each cycle	CPU is completely blocked during the entire burst
Transfer Speed	Slower – one word per stolen cycle	Faster – all data transferred in one go
CPU Efficiency	Higher – CPU runs most of the time	Lower – CPU is halted during burst

→ Explain Cycle stealing in the DMA

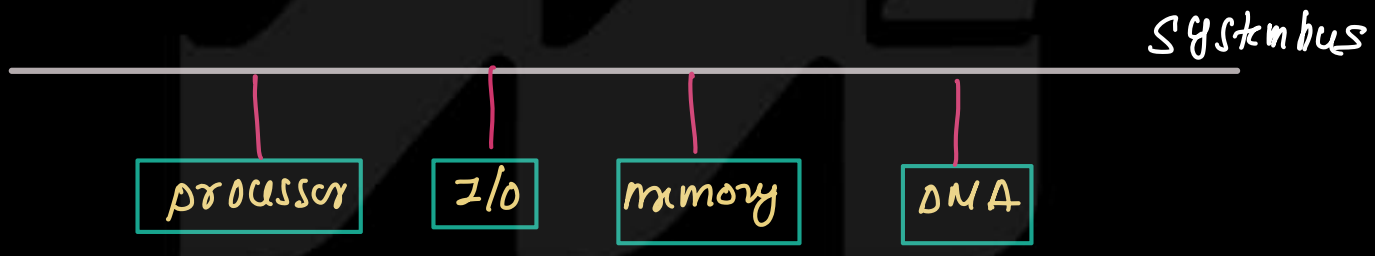
→ Burst Mode – DMA use system Bus completly.

Different DMA Configuration.

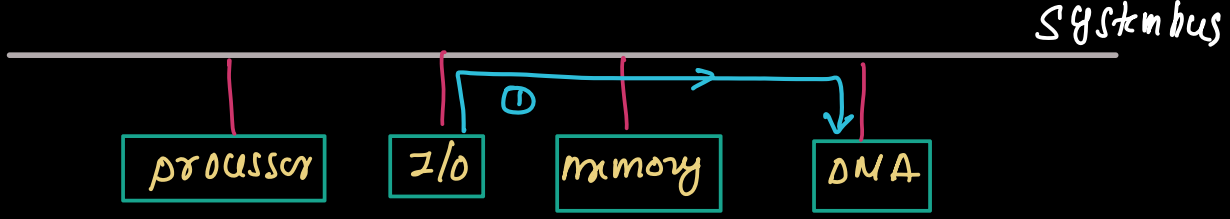
- This represents how components processor, Memory, I/O devices, DMA are connected.
- 3 Configurations are possible.
  1. Single bus detached DMA
  2. Single bus integrated DMA - I/O
  3. I/O Bus

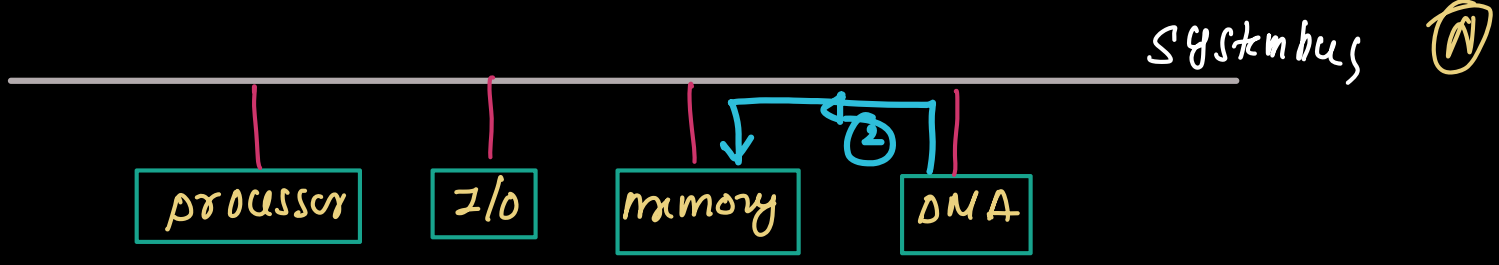
Single bus detached DMA

- In this Configuration processor, memory, I/O devices & DMA are connected to system Bus



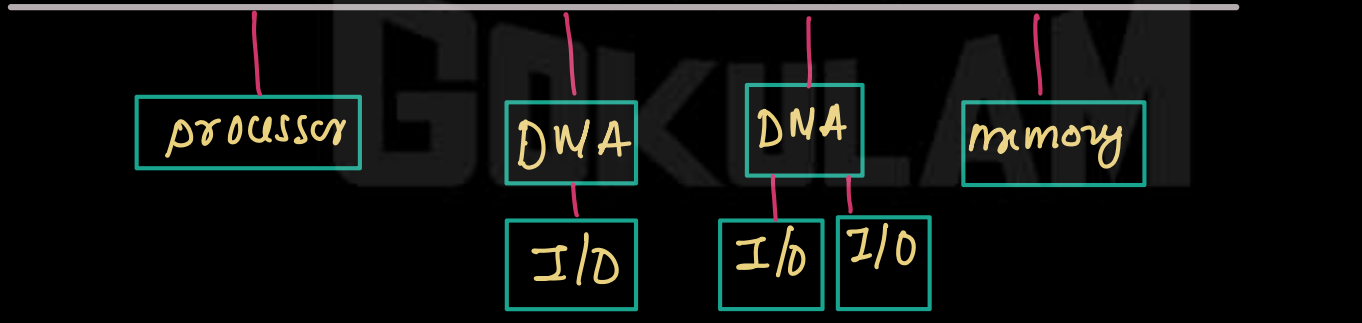
- Consider an I/O operation, read from I/O device and store in memory.
  1. DMA read the data from I/O Module through sys Bus
  2. DMA send data to memory through sys Bus



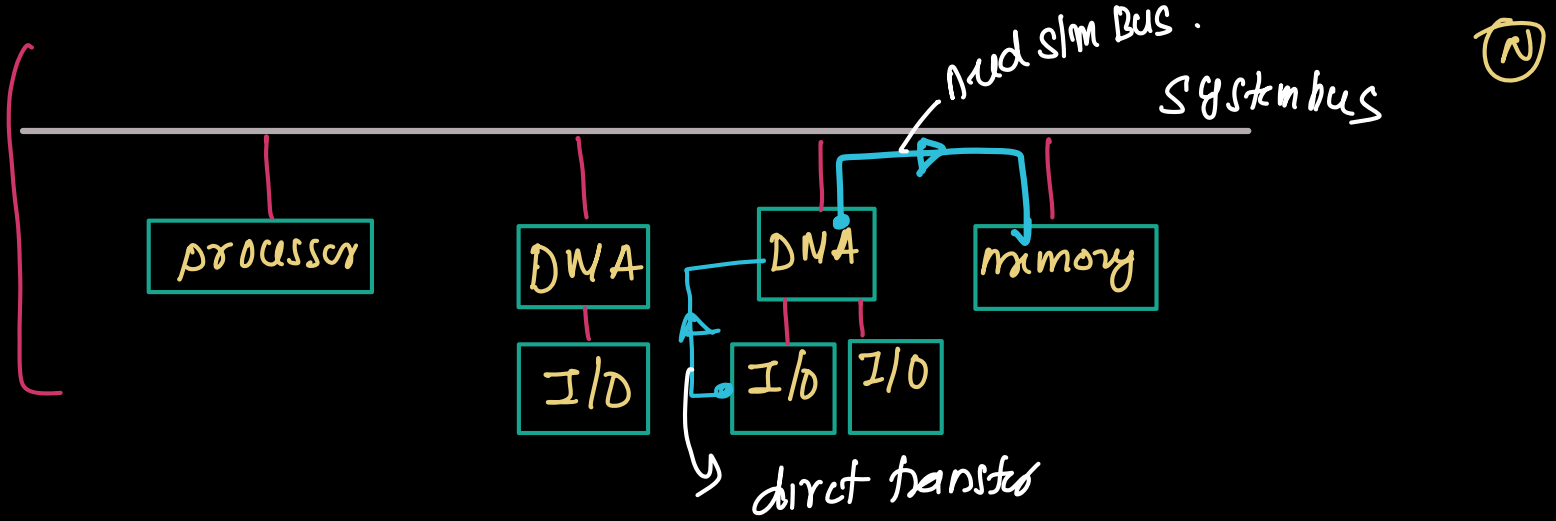


→ System bus is used for the both operation ; so it's a inefficient configuration.

— Single bus with integrated DMA-I/O



→ DMA is integrated with one or more I/O (I/O module)
→ there is direct connection b/w DMA and I/O module
— Data transfer b/w I/O module and DMA occurs directly without using S/m Bus
→ System bus used only when DMA transfer data to memory.



— DMA with I/O Bus

— In this configuration, a separate I/O bus is used to connect multiple I/O devices to the DMA.

→ processor, memory, DMA share system BUS.